DATABASE MANAGEMENT CAPSTONE PROJECT REPORT

Car Rental Management System with Indexing, Views and Advanced SQL Analytics

Submitted in partial fulfilment of the requirement of the degree of

BACHELOR OF TECHNOLOGY

in **CSE with Specialization Data Science (Section - A)**

to

K.R. Mangalam University



by

Ayush Kumar

Roll No: 2501420008

Submission Date: **November 2026**

Under the supervision of

Dr. Swati Gupta

Department of Computer Science and Engineering, School of Engineering and Technology

K.R. Mangalam University, Gurugram – 122001, Haryana, India

1. Introduction

This report describes the database I designed for a small car rental company with three branches (Delhi, Mumbai and Bengaluru). The task was to store cars, customers, rental agreements and rental history, then write the SQL to manage them: CRUD operations, queries for charges, history and availability, an indexing study with EXPLAIN, a couple of views, and some window function queries.

I built everything in SQLite 3.45, driven from Python. SQLite was a practical choice because window functions need version 3.25 or newer and 3.45 comfortably has them. The SQL is standard enough that it moves to MySQL 8 or PostgreSQL with a few changes, which I list in section 8.

The data is made up. It has 3 branches, 5 car categories, 24 cars (25 after the CRUD demo), 18 customers (19 after the demo) and 248 rentals between January and November 2026. Money is in whole rupees. To keep the results repeatable, every query that needs “today” uses the fixed date 2026-11-11 instead of the current date.

Each figure in this report is a console view of one run: the statement that was executed and the result it returned. The complete list of statements is also saved in car_rental_queries.sql, and the finished database is in car_rental.db, so every figure can be reproduced.

2. Database design

2.1 Tables and relationships

I ended up with six tables. A branch owns many cars, a car belongs to one category, and a rental links one customer to one car for a stretch of dates. Maintenance records sit next to the cars so that a car in the garage can be kept out of the available list.

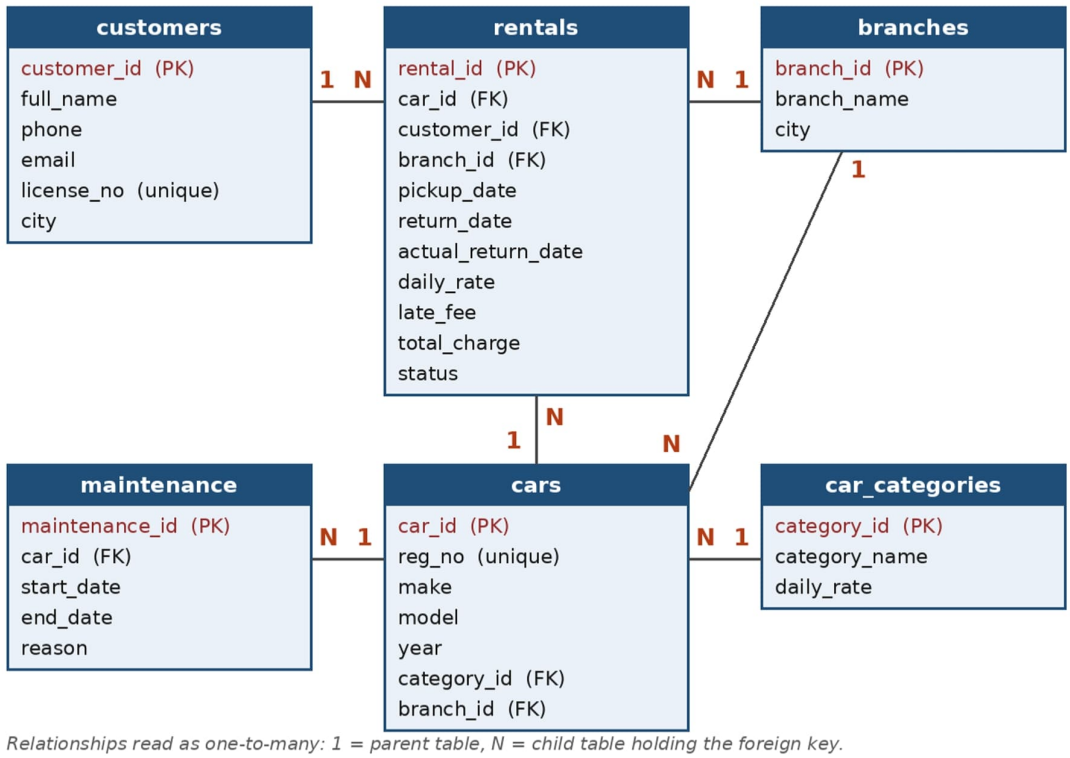


Figure 1: Entity relationship diagram

Table	What it stores	Notes
branches	Pickup and return locations	Three rows: Connaught Place, Andheri West, Indiranagar
car_categories	Economy, Sedan, SUV, Luxury, Van	Holds the standard daily rate for the category
cars	The fleet: registration, make, model, year	Each car has a home branch and one category
customers	Name, phone, email, licence number, city	Licence number is unique
rentals	The rental agreement and its outcome	Dates, rate, late fee, total charge and status
maintenance	Service visits for a car	An empty end_date means the car is still in the garage

A few design choices are worth explaining, because they shaped the queries later on:

- The daily rate lives on the category, but each rental copies it into its own daily_rate column. If the price of an SUV goes up next year, old bills do not change.
- The cars table has no “status” column. Whether a car is free is worked out from the rentals and maintenance tables. I did it this way so the status can never be out of step with the bookings.
- A rental row is the agreement. When the car comes back, actual_return_date, late_fee and total_charge are filled in and the status moves from ACTIVE to COMPLETED. Rental history is therefore just the completed (and cancelled) rows, so I did not need a separate history table.
- SQLite has no date type, so dates are stored as text in YYYY-MM-DD form. That format sorts and compares correctly as plain text.

2.2 Creating the tables

Foreign keys are switched on first. SQLite only checks them when PRAGMA foreign_keys is ON, and the setting has to be repeated for every new connection. The CHECK constraints keep bad data out at the source: a non-positive rate, an unknown status, or a return date that is not after the pickup date.

```
sqlite3 car_rental.db

sqlite> PRAGMA foreign_keys = ON;
sqlite> CREATE TABLE branches (
   ...>   branch_id    INTEGER PRIMARY KEY,
   ...>   branch_name  TEXT NOT NULL,
   ...>   city         TEXT NOT NULL
   ...> );
sqlite> CREATE TABLE car_categories (
   ...>   category_id    INTEGER PRIMARY KEY,
   ...>   category_name  TEXT NOT NULL UNIQUE,
   ...>   daily_rate     INTEGER NOT NULL CHECK (daily_rate > 0)      -- INR per day
   ...> );
sqlite> CREATE TABLE cars (
   ...>   car_id       INTEGER PRIMARY KEY,
   ...>   reg_no       TEXT NOT NULL,                -- unique index is added in section 5
   ...>   make         TEXT NOT NULL,
   ...>   model        TEXT NOT NULL,
   ...>   year         INTEGER CHECK (year >= 2000),
   ...>   category_id  INTEGER NOT NULL REFERENCES car_categories(category_id),
   ...>   branch_id    INTEGER NOT NULL REFERENCES branches(branch_id)
   ...> );
sqlite> CREATE TABLE customers (
   ...>   customer_id  INTEGER PRIMARY KEY,
   ...>   full_name    TEXT NOT NULL,
   ...>   phone        TEXT NOT NULL,
   ...>   email        TEXT,
   ...>   license_no   TEXT NOT NULL UNIQUE,
   ...>   city         TEXT
   ...> );
```

Figure 2: CREATE TABLE statements for branches, car_categories, cars and customers

```
sqlite3 car_rental.db

sqlite> CREATE TABLE rentals (
   ...>   rental_id           INTEGER PRIMARY KEY,
   ...>   car_id              INTEGER NOT NULL REFERENCES cars(car_id),
   ...>   customer_id         INTEGER NOT NULL REFERENCES customers(customer_id),
   ...>   branch_id           INTEGER NOT NULL REFERENCES branches(branch_id),   -- pickup branch
   ...>   pickup_date         TEXT    NOT NULL,
   ...>   return_date         TEXT    NOT NULL,       -- date promised in the agreement
   ...>   actual_return_date  TEXT,                   -- filled in when the car comes back
   ...>   daily_rate          INTEGER NOT NULL,       -- rate frozen at booking time
   ...>   late_fee            INTEGER NOT NULL DEFAULT 0,
   ...>   total_charge        INTEGER NOT NULL DEFAULT 0,
   ...>   status              TEXT    NOT NULL DEFAULT 'BOOKED'
   ...>                       CHECK (status IN ('BOOKED','ACTIVE','COMPLETED','CANCELLED')),
   ...>   CHECK (return_date > pickup_date)
   ...> );
sqlite> CREATE TABLE maintenance (
   ...>   maintenance_id  INTEGER PRIMARY KEY,
   ...>   car_id          INTEGER NOT NULL REFERENCES cars(car_id),
   ...>   start_date      TEXT NOT NULL,
   ...>   end_date        TEXT,                       -- NULL means the car is still in the garage
   ...>   reason          TEXT
   ...> );
```

Figure 3: CREATE TABLE statements for rentals and maintenance

The unique index on the registration number is not in the CREATE TABLE on purpose. I add it in section 5, where the indexing work is shown step by step.

2.3 Sample data

I loaded the tables with a small script that uses a fixed random seed. Rentals for the same car never overlap, a few cars have maintenance windows, and a few rentals came back late. Of the 248 rentals, 228 are completed, 5 are out on the road right now, 7 are future bookings and 8 were cancelled. Three cars are in the garage at the moment.

```
sqlite3 car_rental.db

sqlite> SELECT 'branches' AS table_name, COUNT(*) AS row_count FROM branches
   ...> UNION ALL SELECT 'car_categories', COUNT(*) FROM car_categories
   ...> UNION ALL SELECT 'cars', COUNT(*) FROM cars
   ...> UNION ALL SELECT 'customers', COUNT(*) FROM customers
   ...> UNION ALL SELECT 'rentals', COUNT(*) FROM rentals
   ...> UNION ALL SELECT 'maintenance', COUNT(*) FROM maintenance;
table_name      row_count
--------------  ---------
branches        3
car_categories  5
cars            24
customers       18
rentals         248
maintenance     8

sqlite> SELECT status, COUNT(*) AS rentals FROM rentals GROUP BY status;
status     rentals
---------  -------
ACTIVE     5
BOOKED     7
CANCELLED  8
COMPLETED  228

sqlite> SELECT * FROM cars LIMIT 5;
car_id  reg_no      make     model  year  category_id  branch_id
------  ----------  -------  -----  ----  -----------  ---------
1       DL08RY8629  Maruti   Alto   2024  1            1
2       MH09SG4025  Tata     Tiago  2025  1            2
3       KA08UT4050  Renault  Kwid   2021  1            3
4       DL08JE2485  Honda    City   2025  2            1
5       MH13WU1686  Hyundai  Verna  2025  2            2

sqlite> SELECT rental_id AS id, car_id AS car, customer_id AS cust, pickup_date AS pickup,
   ...>        return_date AS due, actual_return_date AS returned, daily_rate AS rate,
   ...>        late_fee, total_charge AS total, status
   ...> FROM rentals LIMIT 5;
id  car  cust  pickup      due         returned    rate  late_fee  total  status
--  ---  ----  ----------  ----------  ----------  ----  --------  -----  ---------
1   8    4     2026-01-15  2026-01-19  2026-01-19  3500  0         14000  COMPLETED
2   16   10    2026-01-17  2026-01-20  2026-01-20  4200  0         12600  COMPLETED
3   17   7     2026-01-18  2026-01-21  2026-01-21  4200  0         12600  COMPLETED
4   21   11    2026-01-20  2026-01-29  2026-01-29  3500  0         31500  COMPLETED
5   1    15    2026-01-21  2026-01-26  2026-01-29  1500  6750      14250  COMPLETED
```

Figure 4: Row counts and a peek at the cars and rentals tables

3. CRUD operations

3.1 Create

I added a new customer (Tanvi Agarwal, customer 19), a new car (Hyundai Creta, DL01CA7788, car 25) and a four-day booking for 20 to 24 November. The booking is written with INSERT ... SELECT so that the

daily rate comes straight from the category instead of being typed in by hand. Four days at 3,500 gives a charge of 14,000.

```
sqlite3  car_rental.db

sqlite> INSERT INTO customers (full_name, phone, email, license_no, city)
   ...> VALUES ('Tanvi Agarwal', '9818045521', 'tanvi.agarwal@example.com',
   ...>         'DL0420240098765', 'New Delhi');
sqlite> INSERT INTO cars (reg_no, make, model, year, category_id, branch_id)
   ...> VALUES ('DL01CA7788', 'Hyundai', 'Creta', 2024, 3, 1);
sqlite> INSERT INTO rentals (car_id, customer_id, branch_id, pickup_date, return_date,
   ...>                     daily_rate, total_charge, status)
   ...> SELECT c.car_id, cu.customer_id, c.branch_id, '2026-11-20', '2026-11-24',
   ...>        cat.daily_rate, 4 * cat.daily_rate, 'BOOKED'
   ...> FROM cars c JOIN car_categories cat USING (category_id), customers cu
   ...> WHERE c.reg_no = 'DL01CA7788' AND cu.phone = '9818045521';
sqlite> SELECT r.rental_id AS id, cu.full_name AS customer, c.reg_no, r.pickup_date AS pickup,
   ...>        r.return_date AS due, r.total_charge AS total, r.status
   ...> FROM rentals r JOIN customers cu USING (customer_id) JOIN cars c USING (car_id)
   ...> WHERE r.rental_id = (SELECT MAX(rental_id) FROM rentals);
id   customer       reg_no      pickup      due         total  status
---  -------------  ----------  ----------  ----------  -----  ------
249  Tanvi Agarwal  DL01CA7788  2026-11-20  2026-11-24  14000  BOOKED
```

Figure 5: Inserting a customer, a car and a booking

3.2 Read

The first query lists every rental that is active or booked, joined to the customer and the car. The first five rows are cars that are out right now. Rental 233 (Priya Nair, Skoda Slavia) was due back on 9 November and is still out, which I use in the next step. Tanvi’s new booking shows up at the bottom. The second query is a plain lookup by phone number.

```
sqlite3 car_rental.db

sqlite> SELECT r.rental_id AS id, cu.full_name AS customer, c.reg_no,
   ...>        c.make || ' ' || c.model AS car, r.pickup_date AS pickup,
   ...>        r.return_date AS due, r.status
   ...> FROM rentals r
   ...> JOIN customers cu USING (customer_id)
   ...> JOIN cars c USING (car_id)
   ...> WHERE r.status IN ('ACTIVE', 'BOOKED')
   ...> ORDER BY r.pickup_date, r.rental_id
   ...> LIMIT 10;
id   customer        reg_no      car              pickup      due         status
---  --------------  ----------  ---------------  ----------  ----------  ------
233  Priya Nair      DL01QC1975  Skoda Slavia     2026-11-04  2026-11-09  ACTIVE
235  Meera Joshi     MH04HT1492  Hyundai Creta    2026-11-06  2026-11-13  ACTIVE
236  Kavya Menon     KA02SU4252  Maruti Swift     2026-11-06  2026-11-10  ACTIVE
237  Ananya Iyer     MH01PM7511  Maruti Ertiga    2026-11-08  2026-11-14  ACTIVE
238  Farhan Qureshi  MH09GX3145  Volvo S90        2026-11-09  2026-11-18  ACTIVE
239  Kavya Menon     DL04UJ9188  Mahindra XUV700  2026-11-12  2026-11-15  BOOKED
240  Vikram Singh    KA07OU3580  Maruti Ciaz      2026-11-13  2026-11-21  BOOKED
242  Aarav Sharma    MH09SG4025  Tata Tiago       2026-11-20  2026-11-28  BOOKED
243  Ishita Bansal   DL14VA4498  Kia Carens       2026-11-20  2026-11-28  BOOKED
249  Tanvi Agarwal   DL01CA7788  Hyundai Creta    2026-11-20  2026-11-24  BOOKED

sqlite> SELECT customer_id AS id, full_name, phone, email, license_no, city
   ...> FROM customers WHERE phone = '9818045521';
id  full_name      phone       email                    license_no       city
--  -------------  ----------  -----------------------  ---------------  ---------
19  Tanvi Agarwal  9818045521  tanvi.agarwal@example.com  DL0420240098765  New Delhi
```

Figure 6: Open rentals and a customer lookup

3.3 Update

Two updates are shown. First, Tanvi changes her phone number and city. Second, rental 233 is returned on 11 November. That is two days after the due date, so the first UPDATE sets the status, the return date and the late fee, and the second UPDATE recalculates the total. I split it in two because the total depends on the late fee that the first statement has just worked out. The late fee comes to 6,600 (2 days x 2,200 x 1.5) and the total goes from 11,000 to 17,600.

```
sqlite3 car_rental.db

sqlite> UPDATE customers SET phone = '9818099001', city = 'Gurugram'
   ...> WHERE phone = '9818045521';
sqlite> SELECT customer_id AS id, full_name, phone, city FROM customers
   ...> WHERE email = 'tanvi.agarwal@example.com';
id  full_name      phone       city
--  -------------  ----------  --------
19  Tanvi Agarwal  9818099001  Gurugram

sqlite> SELECT rental_id AS id, return_date AS due, actual_return_date AS returned,
   ...>        late_fee, total_charge AS total, status
   ...> FROM rentals WHERE rental_id = 233;
id   due         returned  late_fee  total  status
---  ----------  --------  --------  -----  ------
233  2026-11-09            0         11000  ACTIVE

sqlite> UPDATE rentals
   ...> SET actual_return_date = '2026-11-11', status = 'COMPLETED',
   ...>     late_fee = MAX(0, CAST(julianday('2026-11-11') - julianday(return_date) AS INTEGER))
   ...>                * daily_rate * 3 / 2
   ...> WHERE rental_id = 233 AND status = 'ACTIVE';
sqlite> UPDATE rentals
   ...> SET total_charge = CAST(julianday(return_date) - julianday(pickup_date) AS INTEGER)
   ...>                   * daily_rate + late_fee
   ...> WHERE rental_id = 233;
sqlite> SELECT rental_id AS id, return_date AS due, actual_return_date AS returned,
   ...>        late_fee, total_charge AS total, status
   ...> FROM rentals WHERE rental_id = 233;
id   due         returned    late_fee  total  status
---  ----------  ----------  --------  -----  ---------
233  2026-11-09  2026-11-11  6600      17600  COMPLETED
```

Figure 7: Updating contact details and returning a car

3.4 Delete

A customer with no rentals can be deleted without any fuss, as the test row shows. Trying to delete customer 1 fails, because that customer has rentals and the foreign key refuses to leave them without an owner. That is the behaviour I want. For rentals themselves I would not delete anything in real life. A cancelled booking should be marked CANCELLED so that history and revenue reports stay honest.

```
sqlite3 car_rental.db

sqlite> INSERT INTO customers (full_name, phone, email, license_no, city)
   ...> VALUES ('Test Entry', '9000000000', 'test@example.com', 'TEST000', 'Delhi');
sqlite> DELETE FROM customers WHERE phone = '9000000000';
sqlite> SELECT COUNT(*) AS test_rows_left FROM customers WHERE phone = '9000000000';
test_rows_left
--------------
0

sqlite> DELETE FROM customers WHERE customer_id = 1;
Runtime error: FOREIGN KEY constraint failed (19)
```

Figure 8: A clean delete and a delete blocked by the foreign key

4. Rental charges, history and availability

4.1 Calculating rental charges

The pricing rule I used is simple:

- Base charge = agreed days x daily rate.
- Late fee = days late x daily rate x 1.5, so a late day costs one and a half times a normal day.
- Total = base charge + late fee. Returning early does not earn a refund.

The figure below shows the breakdown for the first few late returns. Rental 28, for example, was a two-day Van booking at 4,200 a day (8,400) returned three days late (3 x 4,200 x 1.5 = 18,900), which gives 27,300.

```
sqlite3  car_rental.db

sqlite> SELECT rental_id AS id, pickup_date AS pickup, return_date AS due,
   ...>        actual_return_date AS returned, daily_rate AS rate,
   ...>        CAST(julianday(return_date) - julianday(pickup_date) AS INTEGER) AS days,
   ...>        CAST(julianday(actual_return_date) - julianday(return_date) AS INTEGER) AS late_days,
   ...>        late_fee, total_charge AS total
   ...> FROM rentals
   ...> WHERE status = 'COMPLETED' AND actual_return_date > return_date
   ...> ORDER BY rental_id
   ...> LIMIT 6;
id  pickup      due         returned    rate  days  late_days  late_fee  total
--  ----------  ----------  ----------  ----  ----  ---------  --------  -----
5   2026-01-21  2026-01-26  2026-01-29  1500  5     3          6750      14250
6   2026-01-22  2026-01-25  2026-01-28  2200  3     3          9900      16500
13  2026-02-06  2026-02-08  2026-02-10  4200  2     2          12600     21000
14  2026-02-08  2026-02-13  2026-02-15  2200  5     2          6600      17600
20  2026-02-12  2026-02-16  2026-02-17  1500  4     1          2250      8250
28  2026-02-19  2026-02-21  2026-02-24  4200  2     3          18900     27300
```

Figure 9: Charge breakdown for late returns

As a sanity check I recalculated the total for every non-cancelled rental from the formula and compared it with the stored value. All 241 rows match. The second query rolls the completed rentals up by category. Luxury cars bring in the most money (1,698,750) from only 38 rentals, while Economy cars have 45 rentals but only 406,500 in revenue.

```
sqlite3 car_rental.db

sqlite> SELECT COUNT(*) AS rentals_checked,
   ...>        SUM(total_charge <> CAST(julianday(return_date) - julianday(pickup_date) AS INTEGER)
   ...>                            * daily_rate + late_fee) AS mismatches
   ...> FROM rentals WHERE status <> 'CANCELLED';
rentals_checked  mismatches
---------------  ----------
241              0

sqlite> SELECT cat.category_name AS category, COUNT(*) AS rentals,
   ...>        SUM(r.total_charge) AS revenue, SUM(r.late_fee) AS late_fees
   ...> FROM rentals r
   ...> JOIN cars c USING (car_id)
   ...> JOIN car_categories cat USING (category_id)
   ...> WHERE r.status = 'COMPLETED'
   ...> GROUP BY cat.category_id
   ...> ORDER BY revenue DESC;
category  rentals  revenue  late_fees
--------  -------  -------  ---------
Luxury    38       1698750  191250
SUV       57       1139250  57750
Van       37       970200   113400
Sedan     52       712800   59400
Economy   45       406500   31500
```

Figure 10: Checking the totals and revenue by category

4.2 Tracking rental history

History is a filter over the rentals table. The first query shows the most recent completed rentals for customer 2, Priya Nair, and flags whether each return was late. Her latest rental is the one we closed in section 3.3, so it shows up as LATE with the 17,600 total.

```
sqlite3 car_rental.db

sqlite> SELECT r.rental_id AS id, c.reg_no, c.make || ' ' || c.model AS car,
   ...>        r.pickup_date AS pickup, r.actual_return_date AS returned,
   ...>        r.total_charge AS charge,
   ...>        CASE WHEN r.actual_return_date > r.return_date
   ...>             THEN 'LATE' ELSE 'on time' END AS return_status
   ...> FROM rentals r JOIN cars c USING (car_id)
   ...> WHERE r.customer_id = 2 AND r.status = 'COMPLETED'
   ...> ORDER BY r.pickup_date DESC
   ...> LIMIT 8;
id   reg_no      car                  pickup      returned    charge  return_status
---  ----------  -------------------  ----------  ----------  ------  -------------
233  DL01QC1975  Skoda Slavia         2026-11-04  2026-11-11  17600   LATE
221  KA11MD7469  Mahindra Marazzo     2026-10-24  2026-10-26  12600   on time
209  KA02JM2094  Toyota Innova Crysta 2026-10-01  2026-10-08  29400   on time
197  DL08RY8629  Maruti Alto          2026-09-19  2026-09-21  3000    on time
179  KA07OU3580  Maruti Ciaz          2026-08-29  2026-09-04  13200   on time
175  KA08UT4050  Renault Kwid         2026-08-20  2026-08-26  9000    on time
156  KA09CW5161  BMW 3 Series         2026-08-05  2026-08-07  15000   on time
148  MH09SG4025  Tata Tiago           2026-07-24  2026-07-27  4500    on time
```

Figure 11: Rental history for one customer

The second query turns it around and shows the history of a single car, the Skoda Slavia DL01QC1975, with the customer who had it each time.

sqlite3 car_rental.db

```
sqlite> SELECT r.rental_id AS id, cu.full_name AS customer, r.pickup_date AS pickup,
   ...>        r.return_date AS due, r.actual_return_date AS returned,
   ...>        r.total_charge AS charge, r.status
   ...> FROM rentals r JOIN customers cu USING (customer_id)
   ...> WHERE r.car_id = (SELECT car_id FROM cars WHERE reg_no = 'DL01QC1975')
   ...> ORDER BY r.pickup_date DESC
   ...> LIMIT 8;
id   customer        pickup      due         returned    charge  status
---  --------------  ----------  ----------  ----------  ------  ---------
233  Priya Nair      2026-11-04  2026-11-09  2026-11-11  17600   COMPLETED
223  Ishita Bansal   2026-10-26  2026-10-30  2026-11-01  15400   COMPLETED
210  Rahul Verma     2026-10-02  2026-10-05  2026-10-05  6600    COMPLETED
190  Arjun Reddy     2026-09-08  2026-09-13  2026-09-16  20900   COMPLETED
168  Aarav Sharma    2026-08-13  2026-08-22  2026-08-22  19800   COMPLETED
154  Karan Malhotra  2026-08-01  2026-08-03  2026-08-04  7700    COMPLETED
134  Vikram Singh    2026-07-09  2026-07-18  2026-07-18  19800   COMPLETED
121  Vikram Singh    2026-06-22  2026-06-27  2026-06-29  17600   COMPLETED
```

Figure 12: Rental history for one car

4.3 Managing vehicle availability

To find cars free for a date range, I check that no live rental and no maintenance window overlaps it. Two periods overlap when each one starts before the other ends. Because the comparison uses strict “less than” and “greater than”, a car coming back on the 15th can go out again on the 15th. The Mahindra XUV700 in the list is due back on 15 November, so it is shown as free for the 15th to 18th. Twenty cars qualify. The five that do not are the three in maintenance, the Volvo S90 that is out until the 18th, and the Maruti Ciaz booked from the 13th. One thing to notice is that the overdue Maruti Swift is in the list, because its agreed return date has passed. A real system would want someone to confirm the car is physically back first.

sqlite3 car_rental.db

```
sqlite> SELECT c.car_id AS id, c.reg_no, c.make || ' ' || c.model AS car,
   ...>        cat.category_name AS category, b.branch_name AS branch
   ...> FROM cars c
   ...> JOIN car_categories cat USING (category_id)
   ...> JOIN branches b USING (branch_id)
   ...> WHERE NOT EXISTS (SELECT 1 FROM rentals r
   ...>                   WHERE r.car_id = c.car_id AND r.status <> 'CANCELLED'
   ...>                     AND r.pickup_date < '2026-11-18' AND r.return_date > '2026-11-15')
   ...>   AND NOT EXISTS (SELECT 1 FROM maintenance m
   ...>                   WHERE m.car_id = c.car_id AND m.start_date < '2026-11-18'
   ...>                     AND COALESCE(m.end_date, '9999-12-31') > '2026-11-15')
   ...> ORDER BY b.branch_id, cat.category_id, c.reg_no;
id  reg_no      car                  category  branch
--  ----------  -------------------  --------  --------------
19  DL06CJ6449  Hyundai i20          Economy   Connaught Place
1   DL08RY8629  Maruti Alto          Economy   Connaught Place
7   DL01QC1975  Skoda Slavia         Sedan     Connaught Place
4   DL08JE2485  Honda City           Sedan     Connaught Place
25  DL01CA7788  Hyundai Creta        SUV       Connaught Place
10  DL04UJ9188  Mahindra XUV700      SUV       Connaught Place
16  DL14VA4498  Kia Carens           Van       Connaught Place
2   MH09SG4025  Tata Tiago           Economy   Andheri West
20  MH07YD3205  Honda Amaze          Sedan     Andheri West
8   MH04HT1492  Hyundai Creta        SUV       Andheri West
11  MH11CO5557  Kia Seltos           SUV       Andheri West
14  MH02SY2768  Audi A4              Luxury    Andheri West
17  MH01PM7511  Maruti Ertiga        Van       Andheri West
18  KA02SU4252  Maruti Swift         Economy   Indiranagar
3   KA08UT4050  Renault Kwid         Economy   Indiranagar
9   KA06OS4200  Tata Nexon           SUV       Indiranagar
21  KA12DA1981  Tata Harrier         SUV       Indiranagar
12  KA09CW5161  BMW 3 Series         Luxury    Indiranagar
15  KA02JM2094  Toyota Innova Crysta Van       Indiranagar
24  KA11MD7469  Mahindra Marazzo     Van       Indiranagar
```

Figure 13: Cars free from 15 to 18 November 2026

Checking availability by hand is fine for searching, but I also wanted the database to refuse a double booking by itself. A BEFORE INSERT trigger runs the same overlap test and aborts the insert if it finds a clash. In the figure, the first booking for Tanvi's Creta (22 to 25 November) is rejected because the car is already booked from the 20th to the 24th. The second one, starting on the 24th, is accepted since the car comes back that day.

```
sqlite3 car_rental.db

sqlite> CREATE TRIGGER trg_no_double_booking
   ...> BEFORE INSERT ON rentals
   ...> WHEN EXISTS (SELECT 1 FROM rentals r
   ...>              WHERE r.car_id = NEW.car_id AND r.status <> 'CANCELLED'
   ...>                AND r.pickup_date < NEW.return_date
   ...>                AND r.return_date > NEW.pickup_date)
   ...> BEGIN
   ...>   SELECT RAISE(ABORT, 'Car already booked for these dates');
   ...> END;
sqlite> INSERT INTO rentals (car_id, customer_id, branch_id, pickup_date, return_date,
   ...>                     daily_rate, total_charge, status)
   ...> VALUES (25, 2, 1, '2026-11-22', '2026-11-25', 3500, 10500, 'BOOKED');
Runtime error: Car already booked for these dates (19)

sqlite> INSERT INTO rentals (car_id, customer_id, branch_id, pickup_date, return_date,
   ...>                     daily_rate, total_charge, status)
   ...> VALUES (25, 2, 1, '2026-11-24', '2026-11-27', 3500, 10500, 'BOOKED');
sqlite> SELECT rental_id AS id, customer_id AS cust, pickup_date AS pickup,
   ...>        return_date AS due, total_charge AS total, status
   ...> FROM rentals WHERE car_id = 25 ORDER BY pickup_date;
id   cust  pickup      due         total  status
---  ----  ----------  ----------  -----  ------
249  19    2026-11-20  2026-11-24  14000  BOOKED
250  2     2026-11-24  2026-11-27  10500  BOOKED
```

Figure 14: A trigger that blocks overlapping bookings

For day to day work the manager mostly wants a headcount. The first query below counts the fleet by branch as in maintenance, rented or available, and the second lists overdue rentals. Right now 18 cars are available, 4 are rented and 3 are in the garage. One rental is overdue: Kavya Menon's Maruti Swift was due on 10 November.

```
sqlite3 car_rental.db

sqlite> SELECT b.branch_name AS branch,
   ...>        SUM(m.car_id IS NOT NULL) AS maintenance,
   ...>        SUM(m.car_id IS NULL AND a.car_id IS NOT NULL) AS rented,
   ...>        SUM(m.car_id IS NULL AND a.car_id IS NULL) AS available,
   ...>        COUNT(*) AS fleet
   ...> FROM cars c JOIN branches b USING (branch_id)
   ...> LEFT JOIN (SELECT DISTINCT car_id FROM maintenance WHERE end_date IS NULL) m
   ...>        ON m.car_id = c.car_id
   ...> LEFT JOIN (SELECT DISTINCT car_id FROM rentals WHERE status = 'ACTIVE') a
   ...>        ON a.car_id = c.car_id
   ...> GROUP BY b.branch_id;
branch           maintenance  rented  available  fleet
---------------  -----------  ------  ---------  -----
Connaught Place  2            0       7          9
Andheri West     1            3       4          8
Indiranagar      0            1       7          8

sqlite> SELECT r.rental_id AS id, c.reg_no, cu.full_name AS customer, r.return_date AS due,
   ...>        CAST(julianday('2026-11-11') - julianday(r.return_date) AS INTEGER) AS days_overdue
   ...> FROM rentals r JOIN cars c USING (car_id) JOIN customers cu USING (customer_id)
   ...> WHERE r.status = 'ACTIVE' AND r.return_date < '2026-11-11';
id   reg_no      customer     due         days_overdue
---  ----------  -----------  ----------  ------------
236  KA02SU4252  Kavya Menon  2026-11-10  1
```

Figure 15: Fleet status per branch and overdue rentals

5. Indexing

Every time someone books a car, the system has to ask whether that car is already taken for those dates. That query filters on car_id with equality and on the dates with ranges, so a composite index on (car_id, pickup_date, return_date) is a natural fit. I compared the query plans before and after creating the indexes using EXPLAIN QUERY PLAN, which is the SQLite version of EXPLAIN.

5.1 Plan before the index

With no index on rentals, SQLite has to read every row in the table (SCAN) to answer the overlap check.

```
sqlite3 car_rental.db

sqlite> EXPLAIN QUERY PLAN
   ...> SELECT * FROM rentals
   ...> WHERE car_id = 7 AND status <> 'CANCELLED'
   ...>   AND pickup_date < '2026-12-05' AND return_date > '2026-12-01';
QUERY PLAN
`--SCAN rentals
```

Figure 16: Overlap check with no index

5.2 Creating the indexes

Four indexes are created: the composite one on rentals, a UNIQUE index on the registration number, and one each on customer phone and email, which are the two things a counter clerk searches by.

```
sqlite3 car_rental.db

sqlite> CREATE INDEX idx_rentals_car_dates ON rentals (car_id, pickup_date, return_date);
sqlite> CREATE UNIQUE INDEX ux_cars_reg_no ON cars (reg_no);
sqlite> CREATE INDEX idx_customers_phone ON customers (phone);
sqlite> CREATE INDEX idx_customers_email ON customers (email);
sqlite> SELECT name AS index_name, tbl_name AS on_table
   ...> FROM sqlite_master WHERE type = 'index' AND sql IS NOT NULL
   ...> ORDER BY tbl_name, name;
index_name            on_table
--------------------  ---------
ux_cars_reg_no        cars
idx_customers_email   customers
idx_customers_phone   customers
idx_rentals_car_dates rentals
```

Figure 17: Creating the indexes and listing them

5.3 Plan after the index

The same query now does a SEARCH using idx_rentals_car_dates. The plan shows car_id=? and pickup_date<? as the search terms: the equality column comes first, then the first range column. The return_date part is only a range as well, and SQLite can use just one range column to narrow the search, so that last condition is checked on the rows that are found. The third column still earns its place. The second query in the figure only needs those three columns, and it is answered from the index alone (COVERING INDEX) without touching the table.

```
sqlite3 car_rental.db

sqlite> EXPLAIN QUERY PLAN
   ...> SELECT * FROM rentals
   ...> WHERE car_id = 7 AND status <> 'CANCELLED'
   ...>   AND pickup_date < '2026-12-05' AND return_date > '2026-12-01';
QUERY PLAN
`--SEARCH rentals USING INDEX idx_rentals_car_dates (car_id=? AND pickup_date<?)
sqlite> EXPLAIN QUERY PLAN
   ...> SELECT COUNT(*) FROM rentals
   ...> WHERE car_id = 7 AND pickup_date < '2026-12-05' AND return_date > '2026-12-01';
QUERY PLAN
`--SEARCH rentals USING COVERING INDEX idx_rentals_car_dates (car_id=? AND pickup_date<?)
```

Figure 18: The same overlap check with the composite index

5.4 Unique index on the registration number

A second car with the registration DL01CA7788 is rejected with a UNIQUE constraint error. That protects the fleet table from duplicates, and the index also makes look-ups by registration number quick.

```
sqlite3 car_rental.db

sqlite> INSERT INTO cars (reg_no, make, model, year, category_id, branch_id)
   ...> VALUES ('DL01CA7788', 'Kia', 'Sonet', 2025, 3, 1);
Runtime error: UNIQUE constraint failed: cars.reg_no (19)
```

Figure 19: The unique index rejecting a duplicate registration number

5.5 Phone and email lookups

Both customer look-ups now use their own index instead of scanning the table.

```
sqlite3 car_rental.db

sqlite> EXPLAIN QUERY PLAN SELECT * FROM customers WHERE phone = '9818099001';
QUERY PLAN
`--SEARCH customers USING INDEX idx_customers_phone (phone=?)
sqlite> EXPLAIN QUERY PLAN SELECT * FROM customers WHERE email = 'tanvi.agarwal@example.com';
QUERY PLAN
`--SEARCH customers USING INDEX idx_customers_email (email=?)
```

Figure 20: Customer look-ups by phone and by email

5.6 A query that does not use the index

An index is not always used. The first query below asks for rentals returned after 1 November, with no car_id. The composite index starts with car_id, so a search on return_date alone cannot use it (the leftmost-prefix rule), and SQLite scans the whole table. The second query looks for phone numbers ending in 4321. Because the pattern starts with a wildcard, the sorted order of the phone index is no help, and that one scans the table too.

```
sqlite3 car_rental.db

sqlite> EXPLAIN QUERY PLAN SELECT * FROM rentals WHERE return_date >= '2026-11-01';
QUERY PLAN
`--SCAN rentals
sqlite> EXPLAIN QUERY PLAN SELECT * FROM customers WHERE phone LIKE '%4321';
QUERY PLAN
`--SCAN customers
```

Figure 21: Two queries where the indexes are ignored

The table below puts the plans side by side.

Query	Without index	With index
Overlap check on one car	SCAN rentals	SEARCH ... USING INDEX idx_rentals_car_dates
Customer by phone	SCAN customers	SEARCH ... USING INDEX idx_customers_phone
Customer by email	SCAN customers	SEARCH ... USING INDEX idx_customers_email
Rentals by return_date only	SCAN rentals	SCAN rentals (index not usable)
Phone LIKE '%4321'	SCAN customers	SCAN customers (leading wildcard)

One honest caveat: with only 248 rentals, both plans finish instantly, so I cannot show a timing difference on this data. The plans are the evidence. On a fleet with years of bookings, the SCAN would read every rental for every booking attempt, while the SEARCH only touches the rows of one car.

6. Views

6.1 Available cars

v_available_cars lists cars that are not out on an active rental and not in the garage. I treat “rented” as having an ACTIVE rental. A car with only a future booking is still available today, which is what a counter clerk would expect. The view returns 18 cars, matching the fleet count in section 4.3.

sqlite3 car_rental.db

```
sqlite> CREATE VIEW v_available_cars AS
   ...> SELECT c.car_id, c.reg_no, c.make, c.model, cat.category_name, cat.daily_rate,
   ...>        b.branch_name
   ...> FROM cars c
   ...> JOIN car_categories cat USING (category_id)
   ...> JOIN branches b USING (branch_id)
   ...> WHERE c.car_id NOT IN (SELECT car_id FROM rentals WHERE status = 'ACTIVE')
   ...>   AND c.car_id NOT IN (SELECT car_id FROM maintenance WHERE end_date IS NULL);
sqlite> SELECT car_id AS id, reg_no, make || ' ' || model AS car, category_name AS category,
   ...>        daily_rate AS rate, branch_name AS branch
   ...> FROM v_available_cars
   ...> ORDER BY branch_name, category_name, reg_no;
id  reg_no      car                  category  rate  branch
--  ----------  -------------------  --------  ----  --------------
2   MH09SG4025  Tata Tiago           Economy   1500  Andheri West
14  MH02SY2768  Audi A4               Luxury    7500  Andheri West
11  MH11CO5557  Kia Seltos           SUV       3500  Andheri West
20  MH07YD3205  Honda Amaze          Sedan     2200  Andheri West
19  DL06CJ6449  Hyundai i20          Economy   1500  Connaught Place
1   DL08RY8629  Maruti Alto          Economy   1500  Connaught Place
25  DL01CA7788  Hyundai Creta        SUV       3500  Connaught Place
10  DL04UJ9188  Mahindra XUV700      SUV       3500  Connaught Place
7   DL01QC1975  Skoda Slavia         Sedan     2200  Connaught Place
4   DL08JE2485  Honda City           Sedan     2200  Connaught Place
16  DL14VA4498  Kia Carens           Van       4200  Connaught Place
3   KA08UT4050  Renault Kwid         Economy   1500  Indiranagar
12  KA09CW5161  BMW 3 Series         Luxury    7500  Indiranagar
9   KA06OS4200  Tata Nexon           SUV       3500  Indiranagar
21  KA12DA1981  Tata Harrier         SUV       3500  Indiranagar
6   KA07OU3580  Maruti Ciaz          Sedan     2200  Indiranagar
15  KA02JM2094  Toyota Innova Crysta Van       4200  Indiranagar
24  KA11MD7469  Mahindra Marazzo     Van       4200  Indiranagar
```

Figure 22: The available cars view

6.2 Customer rental summary

v_customer_rental_summary gives one row per customer with the number of completed rentals, the total spend, and how many returns were late. The status filter is in the LEFT JOIN condition instead of the WHERE clause. That way a customer with no completed rentals, like Tanvi, still appears with zeros instead of vanishing. The top spender is Rohan Gupta with 473,300 over 19 rentals, 5 of them late.

```
sqlite3 car_rental.db

sqlite> CREATE VIEW v_customer_rental_summary AS
   ...> SELECT c.customer_id, c.full_name,
   ...>        COUNT(r.rental_id) AS total_rentals,
   ...>        COALESCE(SUM(r.total_charge), 0) AS total_spend,
   ...>        COALESCE(SUM(r.actual_return_date > r.return_date), 0) AS late_returns
   ...> FROM customers c
   ...> LEFT JOIN rentals r ON r.customer_id = c.customer_id AND r.status = 'COMPLETED'
   ...> GROUP BY c.customer_id, c.full_name;
sqlite> SELECT * FROM v_customer_rental_summary ORDER BY total_spend DESC LIMIT 10;
customer_id  full_name      total_rentals  total_spend  late_returns
-----------  -------------  -------------  -----------  ------------
3            Rohan Gupta    19             473300       5
2            Priya Nair     20             422250       3
5            Vikram Singh   18             388600       2
14           Neha Kapoor    10             387650       1
4            Ananya Iyer    20             372800       1
1            Aarav Sharma   14             341100       4
7            Arjun Reddy    14             328650       2
12           Meera Joshi    12             305100       2
10           Ishita Bansal  16             270650       3
9            Rahul Verma    12             251850       3
```

Figure 23: The customer rental summary view (top 10 by spend)

6.3 Updating through a view

I tried an UPDATE through each kind of view. First a simple view, v_car_fleet, which just selects a few columns from cars. In MySQL and PostgreSQL this is automatically updatable, because every row in the view is exactly one row of the cars table. SQLite is stricter: it treats all views as read-only, so the plain UPDATE fails with “cannot modify v_car_fleet because it is a view”. Adding an INSTEAD OF UPDATE trigger tells SQLite how to pass the change to the cars table, and after that the update works and the base table shows year 2025. This trigger is just doing by hand what MySQL does for free.

```
sqlite3 car_rental.db

sqlite> CREATE VIEW v_car_fleet AS
   ...> SELECT car_id, reg_no, make, model, year, branch_id FROM cars;
sqlite> UPDATE v_car_fleet SET year = 2025 WHERE reg_no = 'DL01CA7788';
Parse error: cannot modify v_car_fleet because it is a view

sqlite> CREATE TRIGGER trg_v_car_fleet_upd INSTEAD OF UPDATE ON v_car_fleet
   ...> BEGIN
   ...>   UPDATE cars SET reg_no = NEW.reg_no, make = NEW.make, model = NEW.model,
   ...>                   year = NEW.year, branch_id = NEW.branch_id
   ...>   WHERE car_id = OLD.car_id;
   ...> END;
sqlite> UPDATE v_car_fleet SET year = 2025 WHERE reg_no = 'DL01CA7788';
sqlite> SELECT car_id AS id, reg_no, make, model, year FROM cars WHERE reg_no = 'DL01CA7788';
id  reg_no      make     model  year
--  ----------  -------  -----  ----
25  DL01CA7788  Hyundai  Creta  2025
```

Figure 24: UPDATE through a simple view, before and after an INSTEAD OF trigger

Then I tried the aggregated view, v_customer_rental_summary. This one fails, and it is a different kind of failure.

```
sqlite3 car_rental.db

sqlite> UPDATE v_customer_rental_summary SET total_spend = 0 WHERE customer_id = 1;
Parse error: cannot modify v_customer_rental_summary because it is a view

sqlite> UPDATE v_customer_rental_summary SET late_returns = 0 WHERE full_name = 'Aarav Sharma';
Parse error: cannot modify v_customer_rental_summary because it is a view
```

Figure 25: UPDATE through the aggregated view fails

The reason is that a row in this view is not a row of any table. total_spend for Rohan Gupta is a SUM over 19 rental rows, and total_rentals and late_returns are a COUNT and a conditional SUM over the same rows. The number 473,300 is not stored anywhere, so there is no cell to write 0 into. Setting it to 0 would mean changing 19 rentals in some unknown way, and the database cannot guess which. MySQL gives ERROR 1288 (“the target table is not updatable”) and PostgreSQL says views containing GROUP BY are not automatically updatable. In SQLite, even adding an INSTEAD OF trigger would not help, because I would have to decide myself what “set the total to zero” means for the underlying rows.

	Simple view (v_car_fleet)	Aggregated view (summary)
Row mapping	One view row = one cars row	One view row = many rentals rows
MySQL / PostgreSQL	Updatable automatically	Not updatable (GROUP BY, SUM, COUNT)
SQLite	Needs an INSTEAD OF trigger	Not updatable; a trigger would have nothing sensible to do
Result in my test	Worked after adding the trigger	Failed

7. Window functions

7.1 RANK(): customers ranked by spend within each branch

The inner query adds up spend per customer per branch. RANK() then numbers the customers inside each branch (PARTITION BY branch) from highest to lowest spend, and the outer query keeps the top three. RANK() leaves a gap after a tie, so two customers tied for first would both get 1 and the next one would get 3. The leaders are Neha Kapoor in Andheri West (245,950), Ishita Bansal at Connaught Place (186,100) and Priya Nair in Indiranagar (200,300). Rohan Gupta is in the top three in two branches.

```
sqlite3 car_rental.db

sqlite> SELECT branch_name, full_name, total_spend, spend_rank
   ...> FROM (
   ...>   SELECT b.branch_name, c.full_name, SUM(r.total_charge) AS total_spend,
   ...>          RANK() OVER (PARTITION BY b.branch_id
   ...>                       ORDER BY SUM(r.total_charge) DESC) AS spend_rank
   ...>   FROM rentals r
   ...>   JOIN customers c ON c.customer_id = r.customer_id
   ...>   JOIN branches  b ON b.branch_id   = r.branch_id
   ...>   WHERE r.status = 'COMPLETED'
   ...>   GROUP BY b.branch_id, c.customer_id
   ...> )
   ...> WHERE spend_rank <= 3
   ...> ORDER BY branch_name, spend_rank;
branch_name      full_name       total_spend  spend_rank
---------------  --------------  -----------  ----------
Andheri West     Neha Kapoor     245950       1
Andheri West     Ananya Iyer     224600       2
Andheri West     Rohan Gupta     188900       3
Connaught Place  Ishita Bansal   186100       1
Connaught Place  Rohan Gupta     185150       2
Connaught Place  Meera Joshi     136500       3
Indiranagar      Priya Nair      200300       1
Indiranagar      Vikram Singh    165400       2
Indiranagar      Karan Malhotra  138300       3
```

Figure 26: Top 3 customers by spend in each branch

7.2 Running monthly revenue per branch

First a CTE adds up the revenue of completed rentals per branch and month, using the month the car came back. Then SUM() OVER (PARTITION BY branch ORDER BY month) turns it into a running total that restarts for each branch. By November the totals stand at 1,990,600 for Andheri West, 1,549,400 for Connaught Place and 1,387,500 for Indiranagar. Andheri West's best month was April (302,300). November is only eleven days old, so its figures are partial.

sqlite3 car_rental.db

```
sqlite> WITH monthly AS (
   ...>   SELECT b.branch_name, substr(r.actual_return_date, 1, 7) AS month,
   ...>          SUM(r.total_charge) AS revenue
   ...>   FROM rentals r JOIN branches b ON b.branch_id = r.branch_id
   ...>   WHERE r.status = 'COMPLETED'
   ...>   GROUP BY b.branch_id, month
   ...> )
   ...> SELECT branch_name, month, revenue,
   ...>        SUM(revenue) OVER (PARTITION BY branch_name ORDER BY month
   ...>                           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
   ...>          AS running_revenue
   ...> FROM monthly
   ...> ORDER BY branch_name, month;
branch_name      month    revenue  running_revenue
---------------  -------  -------  ---------------
Andheri West     2026-01  26600    26600
Andheri West     2026-02  195100   221700
Andheri West     2026-03  183700   405400
Andheri West     2026-04  302300   707700
Andheri West     2026-05  202600   910300
Andheri West     2026-06  102500   1012800
Andheri West     2026-07  291300   1304100
Andheri West     2026-08  163850   1467950
Andheri West     2026-09  247600   1715550
Andheri West     2026-10  176050   1891600
Andheri West     2026-11  99000    1990600
Connaught Place  2026-01  43350    43350
Connaught Place  2026-02  180900   224250
Connaught Place  2026-03  99200    323450
Connaught Place  2026-04  136600   460050
Connaught Place  2026-05  149300   609350
Connaught Place  2026-06  119600   728950
Connaught Place  2026-07  259050   988000
Connaught Place  2026-08  189850   1177850
Connaught Place  2026-09  113100   1290950
Connaught Place  2026-10  218450   1509400
Connaught Place  2026-11  40000    1549400
Indiranagar      2026-01  42000    42000
Indiranagar      2026-02  137150   179150
Indiranagar      2026-03  111700   290850
Indiranagar      2026-04  127400   418250
Indiranagar      2026-05  172800   591050
Indiranagar      2026-06  116700   707750
Indiranagar      2026-07  198600   906350
Indiranagar      2026-08  132000   1038350
Indiranagar      2026-09  150400   1188750
Indiranagar      2026-10  105850   1294600
Indiranagar      2026-11  92900    1387500
```

Figure 27: Monthly revenue and running total per branch

7.3 LAG(): gap between a customer's consecutive rentals

LAG(actual_return_date) gives the return date of the customer's previous rental. Subtracting it from the new pickup date gives the number of days the customer waited before renting again. I used a named window (WINDOW w AS ...) so the definition is written once. The figure shows Priya Nair. Her first rental has no earlier one, so the gap is empty (NULL). After that she rents again within 3 to 12 days, except for one long break of 35 days from 30 April to 4 June.

```
sqlite3 car_rental.db

sqlite> SELECT c.full_name, r.rental_id AS id, r.pickup_date AS pickup,
   ...>        r.actual_return_date AS returned,
   ...>        LAG(r.actual_return_date) OVER w AS prev_returned,
   ...>        CAST(julianday(r.pickup_date)
   ...>             - julianday(LAG(r.actual_return_date) OVER w) AS INTEGER) AS gap_days
   ...> FROM rentals r JOIN customers c USING (customer_id)
   ...> WHERE r.customer_id = 2 AND r.status = 'COMPLETED'
   ...> WINDOW w AS (PARTITION BY r.customer_id ORDER BY r.pickup_date)
   ...> ORDER BY r.pickup_date
   ...> LIMIT 12;
full_name   id   pickup      returned    prev_returned  gap_days
----------  ---  ----------  ----------  -------------  --------
Priya Nair  13   2026-02-06  2026-02-10
Priya Nair  23   2026-02-16  2026-02-25  2026-02-10     6
Priya Nair  36   2026-02-28  2026-03-05  2026-02-25     3
Priya Nair  41   2026-03-09  2026-03-11  2026-03-05     4
Priya Nair  48   2026-03-18  2026-03-20  2026-03-11     7
Priya Nair  59   2026-04-01  2026-04-08  2026-03-20     12
Priya Nair  70   2026-04-15  2026-04-20  2026-04-08     7
Priya Nair  80   2026-04-24  2026-04-30  2026-04-20     4
Priya Nair  110  2026-06-04  2026-06-10  2026-04-30     35
Priya Nair  117  2026-06-19  2026-06-25  2026-06-10     9
Priya Nair  130  2026-06-28  2026-07-05  2026-06-25     3
Priya Nair  136  2026-07-12  2026-07-17  2026-07-05     7
```

Figure 28: Days between consecutive rentals for Priya Nair

7.4 ROW_NUMBER(): most frequently rented car in each category

A first CTE counts rentals per car. A second numbers the cars inside each category by that count, highest first, and the final query keeps row number 1. ROW_NUMBER() always gives exactly one winner per category, even when two cars tie, so I added the registration number as a tie-breaker to make the answer predictable. If ties should all be shown, RANK() is the better function. The Volvo S90 (Luxury) and the Skoda Slavia (Sedan) lead with 12 rentals each, and the other three categories have a leader with 10.

```
sqlite3 car_rental.db

sqlite> WITH car_counts AS (
   ...>   SELECT cat.category_name, c.reg_no, c.make || ' ' || c.model AS car,
   ...>          COUNT(*) AS times_rented
   ...>   FROM rentals r
   ...>   JOIN cars c USING (car_id)
   ...>   JOIN car_categories cat USING (category_id)
   ...>   WHERE r.status IN ('COMPLETED', 'ACTIVE')
   ...>   GROUP BY c.car_id
   ...> ), ranked AS (
   ...>   SELECT *, ROW_NUMBER() OVER (PARTITION BY category_name
   ...>                               ORDER BY times_rented DESC, reg_no) AS rn
   ...>   FROM car_counts
   ...> )
   ...> SELECT category_name AS category, reg_no, car, times_rented
   ...> FROM ranked WHERE rn = 1
   ...> ORDER BY category_name;
category  reg_no      car                  times_rented
--------  ----------  -------------------  ------------
Economy   MH09SG4025  Tata Tiago           10
Luxury    MH09GX3145  Volvo S90            12
SUV       DL04UJ9188  Mahindra XUV700      10
Sedan     DL01QC1975  Skoda Slavia         12
Van       KA02JM2094  Toyota Innova Crysta  10
```

Figure 29: Most rented car in each category

8. Running this on MySQL

The assignment does not say which database to use. If you want to run the same work on MySQL 8, these are the changes I would make:

Topic	SQLite (used here)	MySQL 8
Auto numbering	INTEGER PRIMARY KEY	INT AUTO_INCREMENT PRIMARY KEY
Date types	TEXT in YYYY-MM-DD form	DATE columns
Days between dates	julianday(a) - julianday(b)	DATEDIFF(a, b)
Month of a date	substr(date, 1, 7)	DATE_FORMAT(date, '%Y-%m')
Show the query plan	EXPLAIN QUERY PLAN ...	EXPLAIN ...
Foreign keys	PRAGMA foreign_keys = ON	On by default with InnoDB
Simple view UPDATE	Needs INSTEAD OF trigger	Works automatically
Trigger error	RAISE(ABORT, 'msg')	SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'msg'

9. Conclusion

The finished schema covers customers, cars, branches, rentals and maintenance, and the queries cover everything in the brief. The parts I found most useful were small. A trigger that refuses double bookings means I do not have to trust every piece of application code to check first. Working out availability from the bookings, instead of keeping a status flag, removes a whole class of mismatches. And the EXPLAIN comparison showed clearly why column order matters in a composite index. Next I would add a payments table, security deposits, and a partial index on active rentals.